

ENTREGA FINAL

Data Science III: NLP & Deep Learning aplicado a Ciencia de Datos Flex

Clasificación de Sentimiento sobre IMDB Reviews mediante TF-IDF, PyTorch y Comparación con DistilBERT

Institución:
Coderhouse

Alumno:
Rodolfo Gabriel Riveros Lobos

Fecha:
25/05/2026

1. Introducción y Objetivo

El Procesamiento de Lenguaje Natural (Natural Language Processing, NLP) constituye una de las áreas más relevantes dentro de Ciencia de Datos y Deep Learning, permitiendo transformar información textual no estructurada en conocimiento útil para tareas de clasificación, análisis y automatización.

Dentro de este campo, el análisis de sentimiento representa una aplicación ampliamente utilizada en entornos reales, ya que permite identificar automáticamente la polaridad emocional presente en textos provenientes de reseñas, redes sociales, encuestas o comentarios de usuarios.

El presente proyecto desarrolla un pipeline completo de NLP y Deep Learning aplicado a clasificación binaria de sentimiento utilizando reseñas de películas del dataset IMDB Reviews. El flujo de trabajo incluye exploración de datos, preprocesamiento textual, comparación de técnicas de lematización, representación numérica mediante TF-IDF, construcción de un modelo neuronal en PyTorch y evaluación comparativa frente a un modelo Transformer pre-entrenado basado en DistilBERT.

El objetivo principal consiste en construir y evaluar un sistema capaz de clasificar automáticamente reseñas positivas y negativas, analizando además las fortalezas y limitaciones de las distintas estrategias empleadas.

2. Dataset y Justificación

Para el desarrollo del proyecto se seleccionó el dataset IMDB Reviews, ampliamente utilizado en tareas académicas e industriales de clasificación de sentimiento y análisis textual.

El dataset contiene un total de 50.000 reseñas de películas extraídas de IMDb, distribuidas de forma balanceada entre las dos clases objetivo:

- 25.000 reseñas positivas
- 25.000 reseñas negativas

Esta distribución equilibrada evita sesgos asociados al desbalance de clases y facilita la evaluación objetiva del desempeño de los modelos desarrollados.

Asimismo, el dataset supera ampliamente el requisito mínimo establecido por la consigna, que exige trabajar con al menos 2.000 registros.

La elección de IMDB se justificó por varios factores:

1. Amplia adopción en investigación y enseñanza de NLP.
2. Disponibilidad pública y reproducibilidad.
3. Balance entre clases.
4. Presencia de lenguaje natural real proveniente de usuarios.
5. Adecuación para tareas de clasificación binaria de sentimiento.

Para reducir tiempos computacionales y facilitar la experimentación durante entrenamiento y validación, se trabajó con una muestra estratificada de 10.000 registros (5.000 positivos y 5.000 negativos), preservando el equilibrio original del dataset.

3. Análisis Exploratorio de Datos (EDA)

Antes de iniciar el preprocesamiento y modelado se realizó un Análisis Exploratorio de Datos (EDA) con el objetivo de comprender la estructura del dataset, verificar la distribución de clases y detectar patrones relevantes dentro del texto.

El análisis permitió validar que el dataset presenta una distribución balanceada entre las categorías positiva y negativa, condición favorable para el entrenamiento de modelos supervisados de clasificación.

Además, se realizó un estudio de frecuencia de términos para identificar palabras dominantes y evaluar la presencia de ruido, expresiones repetidas o patrones lingüísticos característicos de cada clase.

3.1 Distribución de Clases

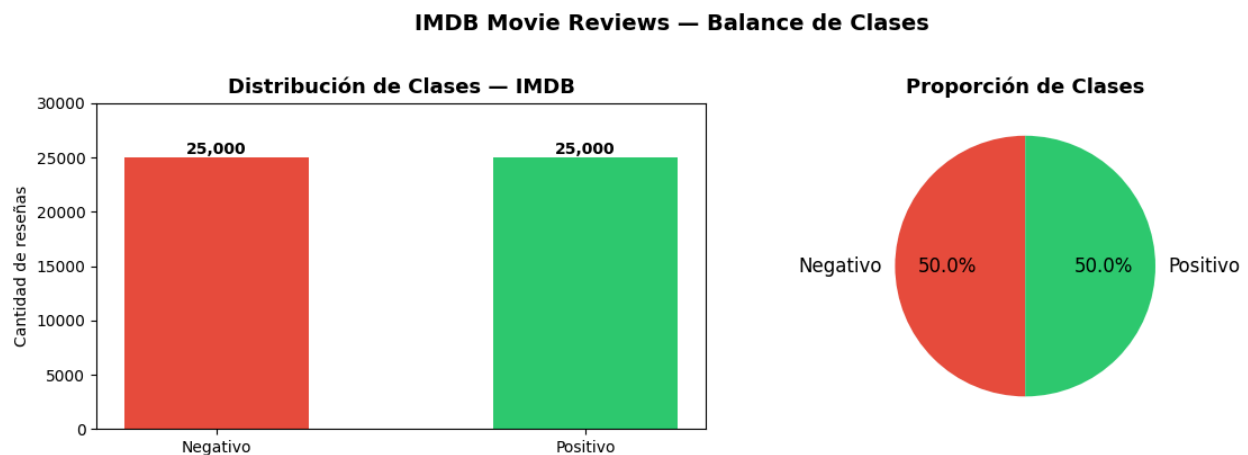
El dataset IMDB utilizado contiene 50.000 observaciones distribuidas de manera equilibrada:

- Sentimiento positivo: 25.000 registros
- Sentimiento negativo: 25.000 registros

La distribución balanceada reduce el riesgo de sesgo hacia una clase dominante y permite interpretar las métricas de evaluación de forma más confiable.

Esta característica elimina la necesidad de aplicar técnicas de balanceo adicionales durante esta etapa del proyecto.

📌 gráfico de barras distribución clases



📌 captura con conteos

```
Total registros : 50,000
Negativo       : 25,000 (50.0%)
Positivo       : 25,000 (50.0%)
Ratio de balance: 1.000
```

3.2 Análisis de Frecuencia de Palabras

Se analizaron las palabras más frecuentes presentes en las reseñas con el objetivo de identificar términos dominantes y comprender la estructura lingüística general del dataset.

Entre las palabras más frecuentes se identificaron términos recurrentes asociados al dominio cinematográfico y a expresiones de valoración emocional. El análisis permitió observar la presencia de palabras frecuentes utilizadas para describir calidad narrativa, actuaciones, experiencia del espectador y percepción general de las películas. Asimismo, la frecuencia elevada de ciertos términos justificó posteriormente la utilización de TF-IDF, técnica que permite reducir el peso de palabras excesivamente repetidas y resaltar aquellas con mayor capacidad discriminativa.

Este análisis permitió detectar:

- Alta presencia de palabras funcionales y conectores.
- Existencia de términos repetitivos asociados al dominio cinematográfico.
- Necesidad de aplicar limpieza textual y lematización.

- Importancia de utilizar técnicas de ponderación como TF-IDF para reducir el impacto de palabras excesivamente frecuentes.

4. Preprocesamiento NLP

El texto original contenido en las reseñas presenta ruido, símbolos especiales, diferencias de formato y variaciones lingüísticas que dificultan el procesamiento automático.

Por este motivo se implementó una etapa de preprocesamiento orientada a normalizar la información antes de la representación numérica y entrenamiento de modelos.

El pipeline aplicado incluyó limpieza mediante expresiones regulares (Regex), tokenización, lematización y comparación entre bibliotecas especializadas de NLP.

4.1 Limpieza Textual mediante Regex

Se aplicaron expresiones regulares (Regex) para eliminar componentes irrelevantes presentes en las reseñas y reducir ruido textual.

Las operaciones realizadas incluyeron:

- Conversión del texto a minúsculas.
- Eliminación de etiquetas HTML.
- Eliminación de caracteres especiales y signos innecesarios.
- Eliminación de números.
- Normalización de espacios múltiples.

Estas transformaciones permitieron obtener un texto más homogéneo y adecuado para etapas posteriores de tokenización y lematización.

ejemplo *antes* / *después* del preprocesamiento

=== EVIDENCIA: ORIGINAL vs LIMPIO ===

--- Registro 0 ---

ORIGINAL : I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was first released in 1967. I also heard th

LIMPIO : i rented i am curious yellow from my video store because of all the controversy that surrounded it when it was first released in i also heard that at

--- Registro 10 ---

ORIGINAL : It was great to see some of my favorite stars of 30 years ago including John Ritter, Ben Gazarra and Audrey Hepburn. They looked quite wonderful. But

LIMPIO : it was great to see some of my favorite stars of years ago including john ritter ben gazarra and audrey hepburn they looked quite wonderful but that w

--- Registro 42 ---

ORIGINAL : WARNING: This review contains SPOILERS. Do not read if you don't want some points revealed to you before you watch the film.

With a cast li

LIMPIO : warning this review contains spoilers do not read if you don't want some points revealed to you before you watch the film with a cast like this you wo

4.2 Tokenización

Posteriormente se realizó la tokenización del texto, proceso mediante el cual cada reseña fue dividida en unidades léxicas individuales (tokens).

La tokenización permitió transformar documentos completos en secuencias analizables, facilitando la posterior aplicación de lematización y vectorización.

Esta etapa resulta fundamental dentro de NLP debido a que constituye el punto de partida para representar el lenguaje natural de forma computacional.

4.3 Comparación entre NLTK y spaCy

Con el objetivo de evaluar distintas estrategias de lematización se compararon las bibliotecas NLTK y spaCy.

NLTK ofrece herramientas ampliamente utilizadas en entornos académicos y permite realizar procesamiento lingüístico clásico mediante reglas y recursos léxicos.

Por otro lado, spaCy incorpora modelos lingüísticos optimizados que permiten una lematización contextual más robusta y eficiente.

La comparación permitió analizar diferencias en precisión, tratamiento contextual y rendimiento computacional.

Característica	NLTK	spaCy
Velocidad	Media	Alta
Lematización contextual	Limitada	Sí
Uso académico	Alto	Alto
Procesamiento por lotes	Limitado	Optimizado

4.4 Selección del Método Final

Luego de la comparación realizada se seleccionó spaCy como herramienta principal para el preprocesamiento definitivo.

La decisión se justificó por:

1. Mayor precisión en la lematización contextual.
2. Mejor rendimiento sobre grandes volúmenes de texto.
3. Procesamiento eficiente por lotes.
4. Integración adecuada con el pipeline posterior basado en TF-IDF.

Esta elección permitió construir representaciones textuales más consistentes antes del modelado.

4.5 Resultado del Preprocesamiento

Como resultado de las etapas anteriores se obtuvo un corpus textual limpio, normalizado y lematizado, preparado para la transformación numérica mediante TF-IDF.

El proceso permitió reducir ruido lingüístico, mejorar consistencia entre documentos y facilitar la extracción de características relevantes para clasificación de sentimiento.

5. Representación Numérica mediante TF-IDF

Luego del preprocesamiento textual fue necesario transformar el corpus en una representación numérica compatible con algoritmos de aprendizaje automático y Deep Learning.

Para esta tarea se utilizó la técnica TF-IDF (Term Frequency - Inverse Document Frequency), implementada mediante `TfidfVectorizer`.

El objetivo de TF-IDF consiste en asignar mayor importancia a palabras con capacidad discriminativa y reducir el impacto de términos excesivamente frecuentes dentro del conjunto documental.

5.1 Configuración del Vectorizador

El vectorizador fue configurado utilizando los siguientes parámetros:

- ``ngram_range=(1,2)``
- ``max_features=10000``
- ``min_df=3``
- ``max_df=0.90``
- ``sublinear_tf=True``

La incorporación de unigramas y bigramas permitió capturar tanto palabras individuales como combinaciones relevantes para la clasificación de sentimiento.

Asimismo, las restricciones aplicadas sobre frecuencia mínima y máxima ayudaron a reducir ruido y eliminar términos poco informativos.

5.2 Reflexión: TF-IDF frente al Conteo de Palabras

Aunque el conteo simple de palabras (Bag of Words) permite representar documentos numéricamente, asigna la misma importancia a todos los términos presentes.

Este enfoque puede generar sesgos debido a palabras extremadamente frecuentes que aportan poca información para la clasificación.

TF-IDF mejora esta representación al:

1. Penalizar palabras excesivamente repetidas.
2. Resaltar términos con mayor capacidad discriminativa.
3. Reducir el efecto del ruido lingüístico.
4. Mejorar la separación entre clases.

En tareas de análisis de sentimiento, esta propiedad resulta especialmente útil porque ciertas palabras poseen una fuerte asociación con polaridad positiva o negativa.

5.3 Resultados de la Vectorización

La aplicación de TF-IDF permitió construir una matriz dispersa representando las características textuales utilizadas posteriormente durante el entrenamiento del modelo neuronal.

El análisis de pesos TF-IDF permitió identificar términos relevantes asociados al sentimiento, observándose palabras con mayor contribución positiva y negativa dentro del vocabulario generado.

La representación TF-IDF generó una matriz de características compuesta por 10.000 atributos ('INPUT_DIM = 10000'), utilizados posteriormente como entrada del modelo neuronal.

El conjunto de entrenamiento quedó representado mediante una matriz de dimensión '7000 × 10000', donde cada fila corresponde a una reseña y cada columna representa un término ponderado según TF-IDF.

Los valores obtenidos presentaron un rango comprendido entre 0.0000 y 0.7278, reflejando la importancia relativa de cada término dentro del corpus.

📌 top palabras TF-IDF positivas

=== TOP 15 FEATURES - POSITIVO ===

film	0.03564
movie	0.03549
good	0.02700
great	0.02316
like	0.02242
time	0.02068
love	0.02063
story	0.02038
see	0.01944
watch	0.01941
character	0.01859
think	0.01737
life	0.01635
find	0.01506
play	0.01503

📌 top palabras TF-IDF negativas

=== TOP 15 FEATURES - NEGATIVO ===

movie	0.04321
film	0.03399
bad	0.03286
like	0.02627
watch	0.02226
good	0.02219
time	0.01992
character	0.01870
see	0.01833
think	0.01820
look	0.01774
scene	0.01672
plot	0.01622
thing	0.01608
story	0.01602

📌 captura matriz o dimensiones

INPUT_DIM : 10,000

NUM_CLASSES : 2

X_train_dense shape : torch.Size([7000, 10000])

y_train shape : torch.Size([7000])

Rango TF-IDF: [0.0000, 0.7278]

6. Modelado Deep Learning

Para la tarea de clasificación de sentimiento se desarrolló un modelo neuronal denominado `TextClassifier`, implementado mediante PyTorch utilizando herencia sobre `nn.Module`.

La arquitectura fue diseñada para procesar vectores TF-IDF de alta dimensionalidad y aprender patrones asociados al sentimiento presente en las reseñas.

6.1 Arquitectura del Modelo

El modelo recibió como entrada vectores TF-IDF de dimensión 10.000 y produjo una salida binaria correspondiente a las clases positiva y negativa.

La arquitectura implementada incluyó:

- Capa entrada: 10.000 características
- Primera capa densa: 512 neuronas
- Segunda capa densa: 256 neuronas
- Tercera capa densa: 128 neuronas
- Capa salida: 2 clases

Además, se incorporaron mecanismos de regularización y estabilización:

- `BatchNorm1d`
- `Dropout (p=0.5)`
- Función de activación ReLU

📌 salida arquitectura PyTorch

```
=== ARQUITECTURA ===
TextClassifier(
  (layer1): Linear(in_features=10000, out_features=512, bias=True)
  (bn1): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (drop1): Dropout(p=0.5, inplace=False)
  (layer2): Linear(in_features=512, out_features=256, bias=True)
  (bn2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (drop2): Dropout(p=0.5, inplace=False)
  (layer3): Linear(in_features=256, out_features=128, bias=True)
  (bn3): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (drop3): Dropout(p=0.5, inplace=False)
  (output): Linear(in_features=128, out_features=2, bias=True)
  (relu): ReLU()
)

Parámetros entrenables: 5,286,786
Device                  : cpu
```

6.2 Complejidad del Modelo

La arquitectura desarrollada presentó un total de 5.286.786 parámetros entrenables.

Este tamaño permitió dotar al modelo de suficiente capacidad para aprender representaciones complejas sobre los vectores TF-IDF generados.

El entrenamiento fue ejecutado utilizando CPU dentro del entorno Google Colab.

6.3 Estrategias de Regularización

Debido a la elevada cantidad de parámetros y al riesgo de sobreajuste, se implementaron mecanismos de regularización durante el entrenamiento.

Se utilizó `Dropout` con probabilidad de 0.5 después de cada bloque denso para reducir dependencia entre neuronas y mejorar generalización.

Asimismo, `Batch Normalization` permitió estabilizar el aprendizaje y acelerar la convergencia del modelo.

7. Entrenamiento del Modelo

El entrenamiento del `TextClassifier` fue realizado utilizando PyTorch sobre la representación TF-IDF obtenida previamente.

Para la optimización de parámetros se utilizó el optimizador `Adam`, mientras que la función de pérdida seleccionada fue `CrossEntropyLoss`, adecuada para tareas de clasificación multiclase y binaria.

Con el objetivo de reducir sobreajuste se incorporó una estrategia de `Early Stopping`, deteniendo el entrenamiento cuando la pérdida de validación no presentara mejoras durante cinco épocas consecutivas.

7.1 Configuración de Entrenamiento

Las principales configuraciones utilizadas durante el entrenamiento fueron:

- Optimizador: `Adam`
- Función pérdida: `CrossEntropyLoss`

- Regularización: `Dropout (p=0.5)`
- Normalización: `BatchNorm1d`
- Estrategia anti sobreajuste: `Early Stopping`
- Paciencia Early Stopping: 5 épocas

7.2 Resultados del Entrenamiento

El mejor desempeño del modelo se obtuvo durante la primera época de entrenamiento.

Los resultados observados fueron:

- Mejor época: 1
- Mejor pérdida validación (`Val Loss`): 0.3019
- Mejor precisión validación (`Val Accuracy`): 0.8793
- Precisión final entrenamiento (`Train Accuracy`): 0.9921

La diferencia entre precisión de entrenamiento y validación evidenció un comportamiento de sobreajuste progresivo, situación controlada mediante Early Stopping.

7.3 Control de Sobreajuste

Durante el entrenamiento se observó un incremento continuo de la precisión sobre entrenamiento mientras que el rendimiento en validación dejó de mejorar.

Para evitar degradación del modelo se activó Early Stopping en la época 6, restaurando automáticamente el mejor estado encontrado.

Este procedimiento permitió conservar el modelo con mejor capacidad de generalización y evitar entrenamiento innecesario.

📌 curvas de aprendizaje

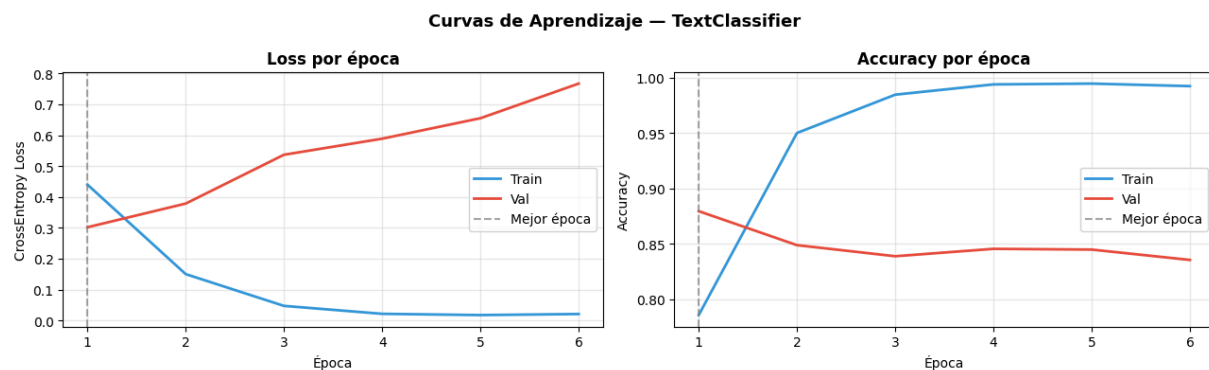


Figura Curvas de Aprendizaje - TextClassifier, se muestra la evolución de entrenamiento y validación. Se observa mejora inicial seguida por indicios de sobreajuste, posteriormente controlados mediante Early Stopping.

8. Resultados

8.1 Evaluación del TextClassifier

El desempeño del modelo desarrollado fue evaluado utilizando métricas clásicas de clasificación, incluyendo Accuracy, Precision, Recall y F1-Score.

Los resultados obtenidos mostraron un comportamiento competitivo para una arquitectura basada en TF-IDF y redes densas implementadas en PyTorch.

Métrica	TextClassifier
Accuracy	0.8700
Precision (macro)	0.87
Recall (macro)	0.87
F1-Score	0.8632

El modelo TextClassifier alcanzó una Accuracy de 0.8700 y un F1-Score de 0.8632 sobre la muestra de evaluación utilizada, mostrando una capacidad adecuada para distinguir reseñas positivas y negativas.

8.2 Comparación con DistilBERT

Métrica	TextClassifier	DistilBERT
Accuracy	0.8700	0.8250
Precision (macro)	0.87	0.83
Recall (macro)	0.87	0.82
F1-Score	0.8632	0.8108

8.3 Interpretación Comparativa

El análisis comparativo mostró un desempeño competitivo entre ambos enfoques.

El TextClassifier obtuvo una Accuracy de 0.8700 y un F1-Score de 0.8632, superando al modelo pre-entrenado DistilBERT, que alcanzó una Accuracy de 0.8250 y un F1-Score de 0.8108.

El análisis de acuerdos indicó:

- Ambos modelos acertaron: 150 observaciones
- Solo TextClassifier acertó: 24 observaciones
- Solo DistilBERT acertó: 15 observaciones
- Ambos modelos fallaron: 11 observaciones

Estos resultados sugieren que la representación TF-IDF combinada con una red neuronal densa puede alcanzar un rendimiento competitivo frente a arquitecturas Transformer pre-entrenadas cuando existe una adecuada estrategia de limpieza, vectorización y regularización.

9. Conclusiones y Mejoras

El presente proyecto permitió desarrollar un pipeline completo de Procesamiento de Lenguaje Natural y Deep Learning aplicado a la clasificación de sentimiento sobre reseñas del dataset IMDB.

A lo largo del trabajo se implementaron etapas de exploración de datos, preprocesamiento textual, representación numérica mediante TF-IDF, modelado con redes neuronales en PyTorch y comparación con un modelo pre-entrenado basado en DistilBERT.

Los resultados obtenidos evidencian que el modelo TextClassifier logró un desempeño competitivo, alcanzando una Accuracy de 0.8700 y un F1-Score de 0.8632, superando ligeramente al modelo pre-entrenado en el conjunto de evaluación utilizado.

Este comportamiento sugiere que una arquitectura relativamente simple, cuando es correctamente diseñada y acompañada de un buen preprocesamiento, puede obtener resultados sólidos en tareas de clasificación de texto.

Sin embargo, también se identificaron limitaciones inherentes al enfoque utilizado, especialmente relacionadas con la representación TF-IDF, la cual no captura relaciones semánticas profundas ni contexto secuencial del lenguaje.

Como mejoras futuras se propone:

- Incorporar embeddings de palabras o embeddings contextuales.
- Implementar modelos basados en Transformers con fine-tuning completo.
- Ampliar el dataset o utilizar el total de registros sin submuestreo.
- Explorar técnicas adicionales de regularización y tuning de hiperparámetros.

En conclusión, el proyecto demuestra la efectividad de combinar técnicas clásicas de NLP con Deep Learning, y evidencia la importancia del preprocesamiento en el rendimiento final de los modelos.

Referencia al Notebook

Todo el pipeline implementado en este proyecto puede ser verificado en el notebook adjunto:

<https://colab.research.google.com/drive/1QnI1fZ0u6mzwwg-G9pYvt1B80ISLAvv3?usp=sharing>

Donde se encuentran:

- Código de preprocesamiento
- Entrenamiento del modelo
- Evaluación de métricas
- Comparación con DistilBERT

El presente documento funciona como informe interpretativo del desarrollo técnico realizado.